INSIGHTFLOW

Revised Architecture Specification

MVP-ready baseline with production requirements, security boundaries, data contracts, and operational controls

Document status

Attribute	Value
Version	2.0 - revised baseline
Date	11 October 2026
Scope	Customer feedback ingestion, AI analysis, issue discovery, and analytics
Decision	Suitable as the basis for an MVP after Priority 0 checks; not production-approved yet
Primary stack	Next.js, FastAPI, Celery, Redis, Supabase PostgreSQL, provider adapters

Architecture decision in one sentence

Keep the overall stack and asynchronous pipeline, but make source collection adapter-based, keep persistent job state in PostgreSQL, isolate browser execution, enforce tenant-aware access controls, and treat AI-provider capability as a validated dependency rather than an assumption.

Prepared as a revised engineering specification. Requirements, scale targets, and provider access that have not been verified are explicitly marked as decisions or gates.

How to use this document

This revision replaces the original high-level five-page description with a buildable architecture baseline. It preserves the original core approach - Next.js, FastAPI, Redis-backed background work, PostgreSQL, and AI-assisted feedback analysis - while making failure handling, data ownership, source access, and security explicit.

Decision area	Current decision	Approval status
Overall component architecture	Retain the modular web API + background-worker design.	Approved as MVP direction
Source collection	Use CSV, configured webhooks, and source-specific API adapters first. Browser automation is opt-in and isolated.	Required for implementation
AI provider	Use an `AnalysisProvider` interface. Select the runtime inference provider only after capability, privacy, cost, latency, and structured-output tests.	Open gate
Adaption Labs	Do not assume it provides a per-feedback sentiment inference endpoint. Validate the actual API contract; use it only for a verified, meaningful workflow.	Critical verification gate
Persistent state	PostgreSQL is authoritative for jobs and business records; Redis is task transport, not the only source of job status.	Approved
Multi-tenancy	Every tenant-owned record is scoped to an organization; authorization is enforced in the API and database access design.	Required before multi-user release
Production readiness	Define capacity, availability, recovery, retention, and cost targets before launch.	Not yet approved

Major changes from the original specification

- Introduces organization and membership boundaries, source connections, durable ingestion jobs, audit records, and issue-to-feedback links.
- Separates sentiment polarity from confidence and adds analysis versions and provenance.
- Defines job states, idempotency, retries, rate limits, error isolation, and a durable queue-dispatch pattern.
- Replaces “bypass blocks” as a strategy with authorized source access and isolated browser execution.
- Adds SSRF safeguards, webhook replay protection, prompt-injection handling, privacy controls, and secrets management.
- Defines API shapes and the meaning of dashboard metrics so UI and backend calculations remain consistent.
- Adds phased delivery, acceptance gates, production-operability requirements, risks, and official technical references.

> **Non-negotiable implementation gate**
>
> Before committing Adaption to runtime scoring, verify an actual endpoint and structured-output contract. Current public docs focus on Adaptive Data and AutoScientist/model-training workflows; they do not prove the per-feedback inference interface assumed here [R1].

1. Goals, scope, and assumptions

1.1 Product goal

InsightFlow ingests customer feedback from permitted sources, normalizes it into a canonical model, analyzes the text, groups recurring complaints into traceable issues, and presents filterable metrics and reports to authorized users.

1.2 In-scope capabilities

- CSV import with validation, row-level errors, deduplication, and asynchronous progress reporting.
- Authenticated webhooks with payload verification and replay/idempotency protection.
- Source-specific native API adapters with pagination, cursors, rate-limit handling, and deletion/update synchronization where the source supports it.
- AI-assisted sentiment, taxonomy assignment, topic extraction, and evidence-backed issue summaries.
- Dashboard metrics, category/trend views, a paginated feedback grid, issue lifecycle management, and asynchronous exports for large results.
- Tenant-aware access control, auditability, privacy controls, and monitoring suitable for an MVP.

1.3 Explicit non-goals for the MVP

- Universal scraping of arbitrary websites or guaranteed access to sources that block or restrict automated collection.
- A large microservices deployment, Kafka, Kubernetes, or a separate analytics warehouse before workload evidence requires them.
- Autonomous high-impact business actions. AI may recommend and summarize; users or deterministic policy approve critical actions.
- Claims of production scalability before representative workload and performance tests establish capacity.

1.4 Assumptions to confirm

Assumption	Why it matters	Required decision
Multi-tenant application	Feedback data is sensitive and must not cross business boundaries.	Confirm organization/workspace model and user roles.
First release source list	Each platform has different access, permissions, limits, and deletion rules.	Name the first 2-3 supported sources.
AI processing contract	Analysis fields and business recommendations depend on the actual provider.	Pass the provider proof-of-capability gate.
Data retention	Customer text may contain personal or confidential information.	Set retention, deletion, export, and raw-payload policy.
Expected load	Worker counts, batching, indexes, and cost limits depend on volume.	Set baseline and peak import sizes, active users, and latency targets.

2. Logical architecture

The design is a modular application with separately operated worker pools. It intentionally avoids splitting every concern into an independent microservice before scale demands that complexity.

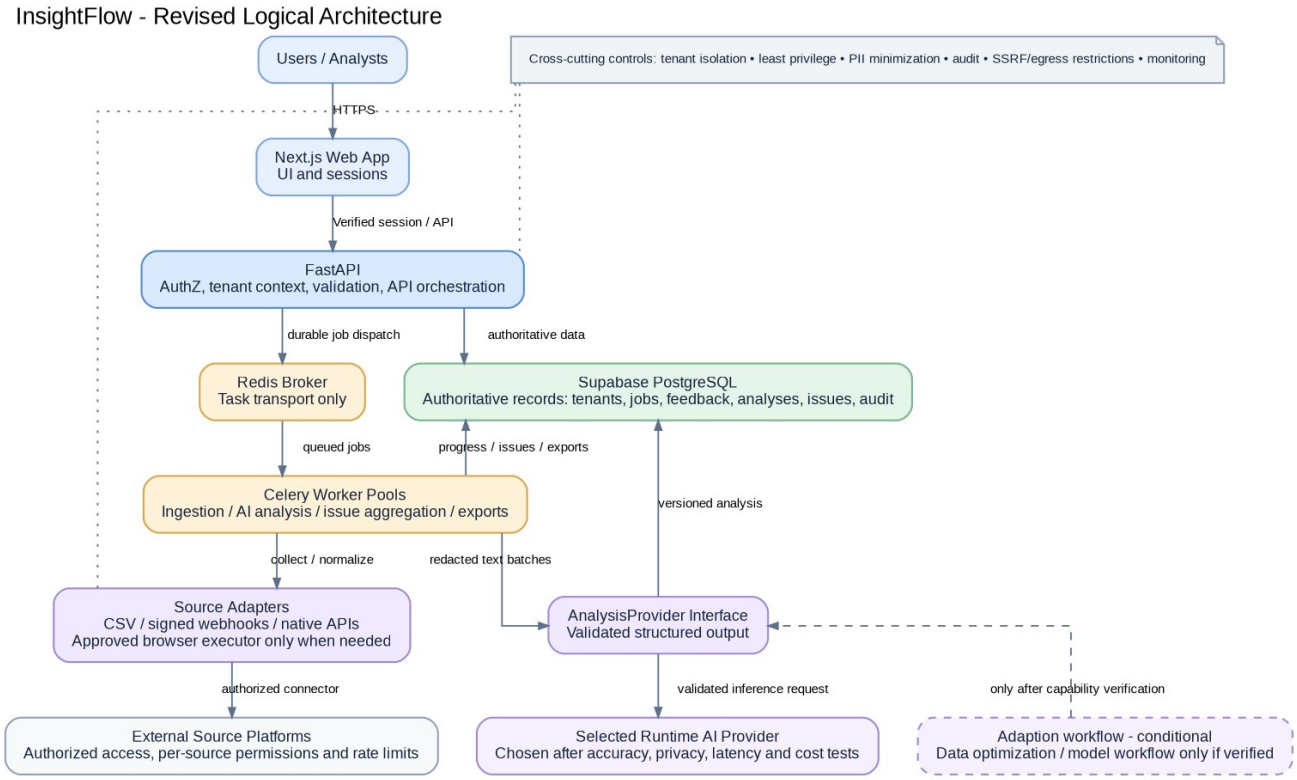


Figure 1. Logical architecture. Dashed Adaption integration is conditional on verified capability and project requirements.

2.1 Component responsibilities

Component	Responsibilities	Must not do
Next.js frontend	Sign-in experience, tenant-aware navigation, filters, visualizations, job progress, issue review, report downloads.	Be the authority for access control or expose server secrets.
FastAPI	Authentication/token validation, authorization, tenant context, input validation, API contracts, job creation, querying, state transitions.	Run long browser/AI workloads in request handlers.
PostgreSQL / Supabase	Authoritative persistence for organizations, source connections, jobs, feedback, analysis versions, issues, audit and outbox data.	Depend on Redis as the only copy of important job/business state.
Redis	Task transport for Celery; optional short-lived cache/rate-limit counters if explicitly designed.	Be treated as the system of record for job lifecycle or source cursors.
Celery workers	Execute ingestion, source sync, analysis, issue aggregation, and export jobs.	Assume exactly-once delivery or retry non-idempotent code blindly.
Source adapters	Handle each source's authentication, pagination, cursor, normalization, rate limits, and policy-specific deletion/update rules.	Assume all source APIs are interchangeable.

Component	Responsibilities	Must not do
Browser executor	Fetch approved web sources when no suitable API exists; run with strict network and resource isolation.	Have database credentials, broad internal network access, or a mandate to bypass controls.
Analysis provider adapter	Call the selected inference service, validate its schema, translate errors, track model/version/cost metadata.	Allow arbitrary model output to alter data or issue status without validation.

2.2 Deployment boundary

Operate the Next.js app, FastAPI API, worker pools, Redis, and PostgreSQL as separate runtime units even if some are initially hosted on the same platform. Browser execution must be isolated from the API and database. The initial deployment can remain small: scale API and worker processes independently as measured demand grows.

3. Canonical data flow

3.1 CSV import

1. The client requests an import and uploads a file within a configurable size limit. FastAPI validates organization access, file type, request size, and import options.
2. The API writes an `ingest_jobs` record and an outbox event in one database transaction, then returns `202 Accepted` with a `job_id`.
3. A dispatcher publishes the job to the Redis/Celery queue and marks the outbox event published. A reconciler re-dispatches durable pending jobs if publication was interrupted.
4. A worker streams/parses the file in bounded chunks, validates rows, records row-level failures, applies privacy controls, and upserts valid feedback records.
5. Analysis jobs are queued only for new or changed text (or when a re-analysis is explicitly requested). Progress and final counts are persisted in PostgreSQL.

3.2 Webhook ingestion

1. Receive only configured webhook routes; identify the source connection and owning organization from server-side configuration, not user-provided organization IDs in the payload.
2. Verify the signature against the raw request body, enforce a timestamp/replay window if supported by the provider, and validate the payload size and schema.
3. Persist a webhook receipt/event with an idempotency key before acknowledging successful receipt. Duplicate deliveries should return a safe acknowledgment without repeating side effects.
4. Normalize the event into the canonical record model and enqueue downstream work. Keep raw payload retention minimized and governed by the data-retention policy.

3.3 Native API collection

1. A user starts a source sync; the API checks permissions and connection health, creates a durable job, and returns immediately.
2. The adapter fetches data page-by-page using provider cursors, respects rate limits and retry headers, and checkpoints its next cursor only after the corresponding batch is durably committed.
3. Each record is normalized, deduplicated, compared for content changes, privacy-processed, and inserted or updated transactionally.

4. Source-supported edits and deletions are reconciled according to the connector policy and applicable source requirements. Unsupported deletion semantics must be documented, not silently ignored.

3.4 Browser-assisted extraction

> **Restricted fallback only**
>
> Browser automation is for explicitly approved source/domain adapters where authorized data access is available and a native API is not adequate. Do not design the product around bypassing bot defenses, login controls, paywalls, CAPTCHAs, or source restrictions.

- Run each browser task in a disposable isolated execution environment with no database credentials and minimal secrets.
- Restrict outbound traffic using an enforceable egress policy; validate URL schemes, DNS resolutions, redirect destinations, and block loopback, private, link-local, and metadata-service destinations. URL-string validation alone is not enough [R5].
- Set hard page count, execution time, memory, download, and navigation limits. Close contexts and remove temporary artifacts after each job.
- Prefer source-specific selectors and parsers. Use an LLM only when necessary to interpret retrieved content; validate extracted records against a strict schema.
- Treat page content as untrusted data and possible prompt injection. Extraction models receive no tools or credentials and cannot issue application actions.

3.5 AI analysis and issue aggregation

1. Send only the minimum required, privacy-processed text and metadata to the configured analysis provider.
2. Validate the response against a versioned schema. Invalid, incomplete, or out-of-range values are retried only when the error is plausibly transient; persistent schema failures become explicit failed analysis records.
3. Persist each analysis version with provider/model, prompt version, taxonomy version, status, timestamps, and distinct sentiment/confidence fields.
4. Run a separate issue-aggregation step to link related feedback, update counts and time windows, generate evidence-backed summaries, and apply explicit priority policy.
5. Critical findings should be reviewable and traceable to supporting feedback. AI-generated recommendations must not be represented as confirmed facts.

4. Data model and integrity rules

The database below is a logical schema baseline. Exact SQL types, enum definitions, migration scripts, and Supabase policies must be committed as versioned migrations before release.

Table	Important fields	Key rules / purpose
organizations	organization_id UUID, name, created_at	Top-level tenant boundary.
organization_memberships	organization_id, user_id, role, status	Maps authenticated users to organizations and allowed roles; user identity may be managed by Supabase Auth.
source_connections	connection_id, organization_id, source_type, secret_ref, config, cursor, status	One source account/configuration per tenant. Store only a secret reference or encrypted credential; never plaintext tokens in ordinary config JSON.

Table	Important fields	Key rules / purpose
ingest_jobs	job_id, organization_id, requested_by, type, status, progress counts, attempts, error_summary, idempotency_key, timestamps	Durable job lifecycle and user-facing progress.
outbox_events	event_id, aggregate_id, event_type, payload, created_at, published_at, attempts	Ensures a committed job can be dispatched even if Redis publication fails.
webhook_events	event_id, connection_id, external_event_id, payload_hash, received_at, processing_status	Deduplicates provider retries and supports safe replay handling. Store the least amount of raw payload necessary.
feedback_entries	feedback_id, organization_id, connection_id, source_external_id, source_created_at, collected_at, text_redacted, text_hash, rating, language, metadata, deleted_at	Canonical feedback. Internal UUID is separate from provider/source identifier.
ai_analyses	analysis_id, feedback_id, analysis_version, provider, model, prompt_version, taxonomy_version, sentiment_label, sentiment_score, confidence_score, primary_category, topics, status, is_current, analyzed_at, error_code	Immutable/versioned analysis results; current analysis is selected explicitly.
issues	issue_id, organization_id, title, description, priority, status, explanation, recommendation, first_seen_at, last_seen_at, supporting_count, owner_id, created_at, updated_at, resolved_at	Issue lifecycle and current aggregate state.
issue_feedback	issue_id, feedback_id, relation_type, relation_score, linked_at	Many-to-many evidence link between issues and supporting feedback.
audit_events	event_id, organization_id, actor_id, action, object_type, object_id, metadata, occurred_at	Records exports, connection changes, role changes, issue status changes, and other sensitive operations.

4.1 Identifier and deduplication rules

- Generate an internal `feedback_id` UUID for every record. Never use a third-party ID as the primary key for the internal data model.
- Preserve the source-provided identifier separately as `source_external_id`.
- For non-null external IDs, enforce uniqueness on the appropriate source scope (recommended: `organization_id + connection_id + source_external_id`). Use a database unique constraint or partial unique index, not an application-only pre-check [R6].
- For CSV rows without a stable external ID, use a documented import-scoped idempotency/deduplication rule. A text hash can assist but should not be the only global identity rule; identical feedback can legitimately occur more than once.
- Use transactions for feedback upsert plus analysis scheduling metadata. Repeated job execution must not create duplicate feedback, analysis-current pointers, or issue links.

4.2 Sentiment and confidence semantics

Field	Meaning	Constraint / rule
sentiment_label	Categorical polarity; recommended values: positive, neutral, negative, mixed, unknown.	Validated enum. Use `unknown` when the model cannot reliably classify.
sentiment_score	Numerical polarity/intensity where supported by the chosen model.	Range -1.0 to +1.0. Do not label it a confidence value.

Field	Meaning	Constraint / rule
confidence_score	Model-reported confidence/probability, if meaningfully supported.	Range 0.0 to 1.0. Do not fabricate if provider does not supply a calibrated value.
primary_category	Organization-defined category label.	Must map to a taxonomy version or approved category set.
analysis_version / taxonomy_version	Defines which model/config and category vocabulary produced the result.	Required for comparisons, reprocessing, and auditability.

4.3 Integrity, indexes, and access rules

- Every tenant-owned table must include `organization_id` directly or be reachable through a constrained, validated relationship to a tenant-owned parent.
- Add foreign keys and check constraints for valid statuses, priorities, ratings, score ranges, and relationship integrity.
- Recommended initial indexes: feedback by `(organization_id, source_created_at DESC)`, current analyses by feedback ID, analyses by `(organization_id, primary_category, sentiment_label)`, issues by `(organization_id, status, priority)`, jobs by `(organization_id, created_at DESC)`, and issue links by both foreign keys.
- Enable and test Row Level Security on any table exposed through the Supabase Data API; use least-privilege grants and never expose service-role/secret keys to the browser [R2].
- If FastAPI uses privileged server credentials, every endpoint must still authorize the user and constrain every query by organization. Privileged credentials must remain server-only; add regression tests for cross-tenant access.

5. API contract baseline

Use a versioned `/v1` prefix, Pydantic request/response models, consistent error envelopes, and generated OpenAPI documentation. Every tenant-owned endpoint derives the organization from the verified user/session context; do not trust a client-provided organization ID without an authorization check.

Method and route	Purpose	Contract requirements
POST /v1/imports	Create a CSV import job.	Validate upload/options; return 202 with `job_id`; idempotency key; row-level errors available from job detail.
POST /v1/webhooks/{source_key}	Receive configured source events.	Verify signature and source connection; enforce payload-size limits; deduplicate; return acknowledgment only after durable receipt.
POST /v1/source-connections	Create/update a source connection.	Role-gated; validate provider configuration; secret references stored server-side; connection-test result.
POST /v1/source-connections/{id}/sync	Start source sync.	Check organization/connection access; create durable job; respect provider sync and rate limits.
GET /v1/jobs/{job_id}	Retrieve job status.	Return status, counts, progress, timestamps, safe error summary, and retryability; authorize job ownership/tenant.
GET /v1/feedback	Search feedback.	Cursor pagination; validated filters; only current/latest analysis by default; redact fields according to role.

Method and route	Purpose	Contract requirements
GET /v1/metrics/summary	KPI and sentiment counts.	Use documented metric semantics and filters; tenant scoping required.
GET /v1/metrics/categories	Category distribution.	Count distinct feedback records with a current successful analysis; make unclassified records explicit.
GET /v1/metrics/trends	Volume/sentiment over time.	Bucket by feedback event time; define fallback for missing source timestamps; specify timezone and interval.
GET /v1/issues	Filter/list issue records.	Filter by status, priority, date range, category/source as supported; cursor pagination.
PATCH /v1/issues/{issue_id}	Update status/owner/priority when permitted.	Validate state transition; audit actor and before/after values; never accept arbitrary AI-generated updates.
POST /v1/exports	Create an export job.	Use the same validated filters as the dashboard; export generated asynchronously for large results.
GET /v1/exports/{export_id}	Check export or download.	Authorize the export owner/tenant; signed short-lived download link if object storage is used; expiry and audit.

5.1 Common API rules

- Use 202 Accepted for work that continues asynchronously. Return a stable `job_id` and a route to check status.
- Use consistent error structure with machine-readable `code`, safe human-readable `message`, optional `field_errors`, `request_id`, and `retryable` flag.
- Use cursor pagination for potentially large result sets; cap page size on the server.
- Validate rating ranges, timestamps, categories, page sizes, export filters, and all IDs at the API boundary.
- Apply request rate limits by user, organization, source, and sensitive operation as appropriate.
- Use request IDs and job IDs across API logs, queue messages, worker logs, and error records.

5.2 Dashboard metric definitions

Metric	Definition for v1	Edge case
Total feedback	Count of distinct feedback entries after tenant and UI filters.	Do not count analysis versions as separate feedback.
Sentiment distribution	Count of records with one current successful analysis per sentiment label.	Show unanalysed/unknown separately; define whether denominator excludes deleted or filtered records.
Category frequency	Count distinct records by current primary category.	Taxonomy version changes must be visible or normalized for trend comparisons.
Trend volume	Bucket feedback by source event timestamp; if absent, use collected timestamp and mark the record as timestamp-estimated.	Use a defined timezone and consistent interval boundaries.
Average sentiment	Only expose if the chosen score is comparable and the UI labels the metric clearly.	Do not average categorical labels or mix model versions without a policy.

Metric	Definition for v1	Edge case
Issue supporting count	Count distinct active feedback records linked to the issue.	Recompute on deletion, dedupe, relink, or source correction.

6. AI analysis architecture

6.1 Provider abstraction

Use an internal adapter boundary so the core domain model is not coupled to a particular model vendor. The exact interface can be implemented in Python; the minimum contract is that input/output schemas are versioned and provider failures map to stable application errors.

```
class AnalysisProvider(Protocol):
    async def analyze_batch(
        self,
        items: list[AnalysisInput],
        taxonomy: TaxonomyVersion,
    ) -> list[AnalysisResult]: ...
```

The provider adapter returns validated sentiment, category, topics, and optional structured explanation fields. It must also report provider/model identifiers and usage metadata where available. The application persists the results and owns all issue lifecycle and authorization decisions.

6.2 Adaption Labs integration gate

The original design assigns per-feedback classification and recommendations to Adaption Labs. Current public Adaption documentation describes Adaptive Data and AutoScientist, including data optimization and model-training workflows [R1]. This does not prove that the exact real-time inference contract required here is available. Keep the integration provider-neutral until confirmed.

Verification item	Evidence required to pass
Supported operation	Exact API/SDK method for the intended task, confirmed against current vendor docs or vendor support.
Input and output	Example request and response validated against the InsightFlow schema, including error cases.
Operational limits	Rate limits, concurrency, batch size, timeout, retry guidance, latency distribution, and availability expectations.
Economics	Unit cost per record/batch and expected cost for peak import volume.
Privacy and retention	Data processing terms, retention behavior, region/residency options if needed, and whether feedback can be sent.
Requirement fit	Written explanation of how Adaption contributes meaningfully to the AI/data workflow requirement.

Fallback decision

If Adaption cannot provide the required real-time inference, use a validated inference provider behind `AnalysisProvider` for runtime scoring. If the project requires Adaption to be used meaningfully, integrate it in a verified data-optimization, dataset preparation, or model-training/evaluation workflow rather than pretending it is the production scoring API.

6.3 Output schema and quality controls

- Use strict JSON/schema validation and permitted enum values. Reject or mark invalid any result that cannot be parsed safely.
- Keep `sentiment_score` and `confidence_score` separate. Only persist confidence if the provider meaningfully defines it; model self-reported confidence is not automatically calibrated probability.
- Use a tenant-owned taxonomy version. If a category is outside the current taxonomy, map to `unknown`/`needs_review` rather than silently inventing a new category.
- Maintain a human-reviewed evaluation set and measure classification quality by source, language, category, and relevant demographic/locale slices when appropriate and lawful.
- Track prompt/config versions and model versions. Reprocessing is an explicit job with deduplication and an auditable version change.
- Do not allow model output to issue SQL, call arbitrary tools, alter permissions, or directly change issue status.
- Minimize text sent to external AI services; redact according to policy and record a provider request ID without logging sensitive prompt content.

6.4 Issue detection is a separate domain process

Individual feedback analysis and issue detection must remain separate. A single negative review is not, by itself, proof of a critical incident. The aggregation service should combine evidence such as frequency, recency, rating, category, and approved severity rules. AI may draft an explanation or recommendation, but the issue record must link to supporting feedback and expose its evidence count and last-seen time.

Priority	Recommended control
Critical	Only based on defined objective triggers or reviewer approval; always show supporting records and why the rule triggered.
High	Evidence threshold or severity rule met; visibly flag uncertainty and allow rapid human review.
Medium	Recurring or materially negative pattern that needs investigation.
Low	Weak or early signal; track without overstating certainty.

7. Security, privacy, and source governance

7.1 Identity and tenant isolation

- Authenticate users using the chosen identity provider (Supabase Auth is the default candidate unless a project constraint dictates otherwise). Validate session/JWT claims server-side in FastAPI.
- Authorize every operation based on user, organization membership, role, object ownership, and state transition. Hiding UI controls is not authorization.
- Require tenant scoping on all reads and writes. Test cross-tenant reads, updates, exports, job access, and issue status changes as part of automated security tests.
- Use least-privilege database roles. Enable RLS on exposed Supabase tables and test policies. Never expose database passwords, AI keys, source credentials, or Supabase service-role/secret keys in the frontend [R2].

7.2 Webhook security

- Verify signatures on the exact raw request body using constant-time comparison where supported by the framework/library.
- Support timestamp validation and replay windows when the source signs timestamps; persist provider event IDs or idempotency keys.
- Enforce payload-size/time limits and reject unsupported content types. Do not log authentication headers or signing secrets.
- Rotate secrets and store them in managed environment secrets or a secret manager; keep secret references separate from ordinary connection configuration.

7.3 Privacy and retention

- Define which text is retained. Default to storing redacted text used for analysis; keep unredacted originals only if a documented business need, access restriction, retention limit, and deletion path exist.
- Regex can catch predictable patterns but is not a complete PII detector. Combine deterministic rules with additional detection appropriate to source and region; allow user review of redaction quality.
- Define deletion and correction workflows across feedback, analyses, issue links, cached aggregates, and exports. Apply source deletion requirements where contractually/legally applicable.
- Set retention periods for original payloads, redacted text, exports, job errors, and audit data. Avoid placing customer text or secrets in application logs.
- Review the privacy terms and retention practices of every external AI and source provider before sending data.

7.4 Safe browser execution and untrusted content

- Use a source/domain allowlist where possible. Restrict browser egress at the network layer and block loopback, private, link-local, multicast, and cloud metadata destinations.
- Resolve DNS and validate every redirect and destination before navigation and subsequent requests; use network-level enforcement to reduce DNS-rebinding and redirect bypass risk [R5].
- Use ephemeral browser contexts and containers; disable unnecessary downloads, permissions, extensions, and persistent profiles. Browser contexts provide session isolation but are not a substitute for OS/container and network isolation [R4].
- Treat retrieved page text as untrusted input. Prompt templates must delimit source content as data, and extraction models must have no application-action tools or secrets.
- Observe source terms, permissions, and provider API limits. The system must fail visibly when a source denies access rather than trying to circumvent the control.

7.5 Audit events

At minimum, audit login/security-relevant events through the identity provider, source connection creation/rotation, imports and exports, member/role changes, issue status or priority changes, and administrator overrides. Audit records should be append-only to ordinary application users, tenant-scoped, and retained according to policy.

8. Reliability, queues, and failure handling

8.1 Job state machine

State	Meaning	Allowed next states
queued	Durably accepted and awaiting worker execution.	running, cancelled, failed

State	Meaning	Allowed next states
running	A worker currently owns an attempt.	succeeded, partially_succeeded, retry_wait, failed, cancelled
retry_wait	Recoverable error; waiting for bounded backoff.	queued, failed, cancelled
partially_succeeded	Some records processed and some failed.	running (explicit retry), succeeded, failed
succeeded	Completed according to defined success rules.	No automatic transition; reprocessing creates a new job.
failed	Permanent error or retry limit exceeded.	Explicit retry/new job only.
cancelled	Cancelled by an authorized request where safe.	Terminal after worker cleanup.

Store transitions and progress in PostgreSQL. A task message carries identifiers and minimal parameters; the worker reloads authoritative state. User-visible progress must be best-effort and must not rely only on Celery's transient result backend.

8.2 Queue and worker separation

Queue / pool	Workload	Isolation / limit
ingest	CSV parsing, webhook normalization, API source synchronization.	Bounded chunk size; per-source limits; CPU/memory caps.
ai	External inference calls and output validation.	Provider concurrency, batch and token limits; provider-specific rate limits.
browser	Browser-based source collection.	Separate isolated execution environment; strict egress; low concurrency; hard timeout.
aggregation	Issue linking, issue summaries, trend/aggregate refresh if precomputed.	Idempotent recomputation; tenant-aware batches.
exports	Filter, generate, and stage downloadable exports.	Asynchronous for large data; short-lived download access; cleanup policy.

8.3 Retry and idempotency policy

- Use Celery as the initial worker framework rather than leaving a Celery/RQ choice undecided. Configure retries by error class, exponential backoff and jitter for transient errors, and maximum attempts [R3].
- Do not retry permanent validation failures, revoked credentials, forbidden source access, or unsupported response schemas without intervention.
- Design tasks to be idempotent. Assume at-least-once execution/redelivery can occur; never claim exactly-once processing. Celery documentation explicitly recommends idempotent task functions for safe redelivery [R3].
- Persist per-record outcomes so a retry processes failed/retryable items without repeating successful side effects.
- Add a reconciler that detects stale `queued`/`running` jobs, unpublished outbox events, and jobs with no heartbeat beyond a configured threshold.
- Define cancellation semantics: stop scheduling new items, allow safe in-flight work to finish or terminate, and record what has already been committed.

8.4 Failure matrix

Failure	Expected behavior	User/ops visibility
Redis unavailable	API can persist a pending job/outbox row; dispatcher retries later. Do not claim queued work started.	Job remains queued; alert on queue publication delay.
Worker crash	Task is retried/reconciled based on idempotency and retry policy; committed records are not duplicated.	Job attempt count and safe error summary.
AI rate limit / timeout	Respect Retry-After where available, back off, cap retries, and keep records pending/failed without corrupting ingestion.	Analysis status and provider error class; cost/latency metrics.
Invalid AI JSON	One bounded repair/retry if appropriate; otherwise mark analysis failed/needs_review.	Schema validation error; no silent coercion.
Source API permission denied	Stop retries until credentials/permissions are fixed.	Connection unhealthy with actionable error.
Bad CSV rows	Continue valid rows and report rejected rows and reasons.	Processed/rejected counts and downloadable error report.
Database unavailable	Do not acknowledge durable intake or mark job complete; apply bounded retries and alert.	Dependency health and queue backlog.

9. Performance, scaling, and observability

9.1 Scale by workload, not by fashion

Start with a modular deployment and separate processes for the API and each worker pool. Scale each independently using queue depth, processing latency, and resource utilization. Do not add Kafka, Kubernetes, a data warehouse, or additional microservices until load tests or operational limits justify them.

9.2 Performance controls

- Use bounded batches for CSV parsing, database upserts, and AI inference. Avoid placing tens of thousands of individual tasks into Redis from a single synchronous request.
- Use connection pooling and database indexes for common organization-scoped filters and chronological queries.
- Use cursor-based pagination and server-side filter limits. Never fetch all feedback to the browser for filtering.
- Measure dashboard query plans before adding caching or materialized aggregates. If query cost grows, introduce precomputed tenant-scoped daily aggregates with a defined freshness SLA.
- Generate large exports asynchronously. Avoid holding a database transaction open while a file is downloaded.
- Configure per-provider request/concurrency budgets and global job fairness so one tenant cannot monopolize workers.

9.3 Required observability

Signal	Examples / dimensions	Why it matters
API	Request rate, p50/p95/p99 latency, 4xx/5xx, auth failures; route and tenant tier.	Detect user-visible degradation and abuse.
Ingestion	Records received, accepted/rejected, dedupe count, source lag, cursor progress.	Detect source drift, bad imports, and stalled sync.
Queues/workers	Queue depth/age, task duration, retries, failures, worker utilization, heartbeat age.	Find backlogs and worker starvation.
AI	Latency, error class, schema-invalid output rate, tokens/cost where available, analysis coverage.	Control quality and cost; diagnose provider issues.
Database	Connection use, slow queries, locks, storage, backup status, restore-test status.	Maintain query and recovery health.
Security	Webhook signature failures, SSRF block events, authorization denials, export volume, role changes.	Detect abuse and validate controls.

10. Deployment and operational requirements

10.1 Environment layout

Environment	Minimum expectation
Development	Local/test credentials, synthetic feedback, migrations, lint/unit tests, local Redis/PostgreSQL or non-production managed resources.
Staging	Separate database/project and provider keys; representative volume tests; webhook signature tests; end-to-end import-to-dashboard verification.
Production	Separate credentials and data; secret manager/environment secrets; controlled schema migrations; backups and restore testing; alerting; access audit.

10.2 Deployment topology

- Frontend: Next.js hosted as a web app with secure session handling and environment-specific configuration.
- API: stateless FastAPI processes behind HTTPS, health/readiness probes, request limits, and rate limiting.
- Worker pools: separate deployable processes for ingestion, AI, aggregation, exports, and browser execution. Browser tasks run in isolated compute/network boundaries.
- Data services: managed PostgreSQL/Supabase and Redis with private networking where supported, least-privilege credentials, monitoring, and backups appropriate to the chosen plans.
- CI/CD: automated tests and migration checks; deploy to staging first; maintain a rollback plan for application releases and a forward/backward-compatible migration plan.

10.3 Non-functional targets that must be decided

> **Do not invent capacity commitments**
>
> The architecture cannot be called scalable until the team defines actual targets and tests them. Fill in the values below from expected product usage, then validate with load and failure testing.

Target	Required value / decision
Maximum CSV size and rows per import	To be set
Expected normal and peak feedback volume per tenant	To be set
Concurrent active users and simultaneous imports	To be set
API p95 latency target for interactive routes	To be set
Target time to analyze a defined import size	To be set
Maximum queue age before alert	To be set
Recovery Point Objective (RPO) and Recovery Time Objective (RTO)	To be set
Data retention and export expiry periods	To be set
Monthly AI/provider budget and per-tenant usage caps	To be set
Supported languages, regions, and data residency constraints	To be set

11. Phased implementation plan

Phase	Deliverables	Exit gate
Phase 0 - proof and decisions	Confirm Adaption requirement/API capability; select runtime inference provider; choose first sources; define tenant model, retention policy, and capacity targets.	Provider contract proven with sample inputs/outputs; source permissions and access paths are documented; unresolved critical decisions have owners.
Phase 1 - reliable MVP core	Authentication and organization model; CSV import; durable jobs/outbox; canonical schema; deterministic preprocessing; one validated analysis provider; basic dashboard.	Repeat-import idempotency, job recovery, tenant-isolation, and end-to-end tests pass.
Phase 2 - source integrations	One or two native API adapters; configured webhook path; cursor/resume, retry and source deletion/update behavior.	Connector contract tests and provider limit/failure tests pass.
Phase 3 - issue intelligence	Taxonomy management; current/versioned analyses; issue aggregation; evidence links; status lifecycle; human review of priority recommendations.	Every issue is traceable to evidence; evaluation set and human correction flow are available.
Phase 4 - production hardening	Browser isolation if needed; exports; monitoring/alerts; backups/restore; load and security testing; documentation and runbooks.	NFR thresholds, recovery tests, security checks, and operational alert drills pass.

11.1 Minimum acceptance checklist

- A user from Organization A cannot read, update, export, or poll jobs belonging to Organization B.
- Replaying the same import or webhook does not duplicate feedback or duplicate issue-support links.
- An interrupted job can resume or be retried without repeating already-committed side effects.
- AI provider timeout, rate limit, malformed output, and unavailability produce clear and bounded outcomes.
- A source connection with expired/invalid credentials fails visibly and does not loop through endless retries.
- A browser fetch cannot reach prohibited internal destinations; this is tested at the network boundary, not only in unit tests.
- Dashboard counts match seeded known data and follow the metric definitions in Section 5.2.
- Deleting/withdrawing a feedback record follows the documented retention and issue/aggregate update rules.
- Restore from database backup has been tested against the agreed RPO/RTO.
- Exports are tenant-scoped, auditable, time-limited where applicable, and do not expose internal storage paths.

12. Risk register and architecture decisions

Risk	Impact	Mitigation / owner action	Priority
Adaption runtime inference assumption is wrong or incomplete	Core AI workflow must be redesigned late.	Complete provider spike in Phase 0; keep runtime scoring behind AnalysisProvider.	Critical
Target review source has no authorized/usable access path	Data collection feature fails or violates source rules.	Limit MVP to sources with confirmed APIs, approved webhooks, or authorized imports.	Critical
Cross-tenant data leakage	Severe privacy and business trust incident.	Tenant-aware schema, authorization at every endpoint, RLS/grants as applicable, automated negative tests.	Critical
Browser worker reaches internal/cloud metadata destinations	Infrastructure compromise or credential exposure.	Network egress controls, isolated workers, redirect/DNS checks, no DB credentials, security testing.	Critical
AI hallucinates a critical issue or recommendation	Incorrect business action and loss of trust.	Evidence links, objective thresholds, uncertainty handling, human review, evaluated model output.	High
At-least-once job execution duplicates data	Incorrect analytics and repeated issue creation.	Idempotent handlers, unique constraints, durable job/outbox model, replay tests.	High
Cost/latency grows with large imports	Slow dashboard updates or unsustainable unit economics.	Batching, queue limits, provider budgets, asynchronous processing, load tests.	High
PII leaks to logs or third-party AI	Privacy and compliance risk.	Data minimization, redaction, secret scrubbing, restricted logs, provider review, retention policy.	High

12.1 Architecture decision records (proposed)

ADR	Decision	Rationale / consequence
ADR-001	Keep modular API + background workers; defer microservices.	Matches current complexity and keeps deployment operationally manageable.

ADR	Decision	Rationale / consequence
ADR-002	Celery is the initial task framework; Redis is the broker.	Removes framework ambiguity; requires idempotency and explicit retries.
ADR-003	PostgreSQL is authoritative for business and job state; Redis is transport.	Jobs survive queue-state loss and can be reconciled.
ADR-004	All external feedback connectors implement a canonical adapter contract.	Source-specific complexity does not leak into domain and UI logic.
ADR-005	Browser automation is an isolated fallback for approved sources, not a universal scraper.	Reduces security, compliance, and reliability risk.
ADR-006	AI provider is replaceable through a validated structured-output contract.	Prevents the domain from depending on an unverified vendor capability.
ADR-007	Issues are separate aggregated entities linked to supporting feedback.	Provides explainability, lifecycle tracking, and repeatable aggregation.

13. Open decisions before production sign-off

Decision / question	Default recommendation	Owner / status
Which sources are guaranteed in v1?	Choose two or three with confirmed access and stable integration paths.	Product + engineering / Open
What exactly must Adaption contribute?	Confirm requirement wording and documented capability; do not assume runtime sentiment inference.	Project lead / Critical
Which runtime model/provider?	Select based on tested accuracy, structured output, privacy terms, cost, rate limits, and region.	AI owner / Open
Can original unredacted text be stored?	Default no unless an explicit use case and retention/access controls justify it.	Privacy/product / Open
What roles exist?	Start with organization owner/admin, analyst, and read-only viewer unless simpler MVP roles suffice.	Product / Open
What are deletion obligations?	Document source-driven deletion, tenant deletion, export cleanup, and backup retention semantics.	Legal/product/engineering / Open
What are the NFR values?	Set workload, latency, recovery, retention, and budget targets before production.	Product/operations / Open
Where is this deployed?	Pick managed services compatible with worker processes, networking isolation, secrets, and backup requirements.	Engineering / Open

14. Final recommendation

> **Architecture review decision**
>
> APPROVE AS AN MVP DESIGN BASELINE, with mandatory completion of the Phase 0 gates. DO NOT approve as production-ready until tenant isolation, authorized source access, the AI provider contract, browser isolation where applicable, recovery/observability, and capacity targets have passed their tests.

15. References

Official documentation consulted for specific implementation guidance. Provider capabilities and service terms should be rechecked when implementation begins; these references are not a substitute for source-specific access approval or contract review.

[R1] Adaption - Getting started / current product scope - https://docs.adaptionlabs.ai/introduction/getting-started

Current documentation describes Adaptive Data and AutoScientist and is the basis for treating per-feedback inference as an unverified requirement.

[R2] Supabase - Row Level Security and secure data - https://supabase.com/docs/guides/database/postgres/row-level-security

RLS, grants, least privilege, and protection of service-role credentials.

[R3] Celery - Tasks, retries, and idempotency - https://docs.celeryq.dev/en/latest/userguide/tasks.html

Task acknowledgment, retry behavior, exponential backoff, and idempotent task guidance.

[R4] Playwright - Browser contexts - https://playwright.dev/docs/browser-contexts

Browser context/session isolation; not a replacement for OS/container/network isolation.

[R5] OWASP - SSRF Prevention Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

URL handling, allowlists, redirect risks, internal address ranges, and network protections.

[R6] PostgreSQL - Constraints - https://www.postgresql.org/docs/current/ddl-constraints.html

Primary keys, foreign keys, check constraints, and composite uniqueness.

[R7] Trustpilot Developer Documentation - https://developers.trustpilot.com/

Example of a source with distinct API access models and authentication requirements; exact entitlement depends on product/account.

Appendix A. Canonical analysis result example

Illustrative schema only; numeric values are placeholders, not model performance claims. The provider adapter must validate output and map vendor-specific fields before persistence.

```
{
  "feedback_id": "<internal UUID>",
  "analysis_version": 1,
  "provider": "<validated-provider-adapter>",
  "model": "<model identifier>",
  "prompt_version": "feedback-analysis-v1",
  "taxonomy_version": "<tenant taxonomy version>",
  "sentiment_label": "negative",
  "sentiment_score": -0.78,
  "confidence_score": 0.86,
  "primary_category": "payments",
  "topics": ["checkout", "payment failure"],
  "status": "succeeded",
  "analyzed_at": "<RFC 3339 timestamp>"
}
```

Appendix B. Recommended initial repository boundaries

```
apps/
  web/                  # Next.js UI
  api/                  # FastAPI routes, authz, schemas
workers/
  ingest/               # CSV and source adapter tasks
  ai/                   # provider adapters and validation
  aggregation/          # issue aggregation and evidence links
  exports/              # report generation
  browser_executor/     # isolated browser runtime; no DB secrets
packages/
  contracts/            # versioned schemas and enums
  domain/               # shared domain logic, policies
  observability/        # logging, metrics, correlation IDs
migrations/             # versioned PostgreSQL changes
tests/
  unit/ integration/ e2e/ security/ load/
```

This is a logical boundary proposal, not a requirement to create a separate deployable service for every folder. Keep the repository layout proportional to team size and deployment complexity.